

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ingeniería Electrónica, Mecatrónica y Biomédica

Electrónica Digital 2

Proyecto 1

Car Wash Inteligente

Monitoreo de una red de sensores

Integrantes: Miguel Donis - Ian Farrington

Carnés: 22993 - 21952

Catedrático: Pablo Mazariegos

Fecha: 11/08/2026

Índice

1. Introducción	1
2. Objetivos	1
2.1. Objetivo general	1
2.2. Objetivos específicos	2
3. Descripción general del proyecto	2
4. Funcionamiento del sistema	3
5. Arquitectura del sistema y distribución de microcontroladores	4
5.1. ATmega328P Maestro	4
5.2. ATmega328P Slave 1 – dirección I2C 0x11	4
5.3. ATmega328P Slave 2 – dirección I2C 0x12	4
5.4. Bus I2C compartido	5
5.5. Alimentación	5
6. Red de sensores	5
6.1. Sensor infrarrojo de entrada	5
6.2. HC-SR04 y nivel del depósito	6
6.3. VL53L0X	6
7. Actuadores	6
7.1. Servomotores	6
7.2. Motor DC y L298N	7
7.3. Motor paso a paso y ULN2003	7
8. Pantalla LCD	7
9. Protocolos de comunicación	8

9.1. I2C	8
9.2. UART	8
10.Diseño y construcción de la maqueta	9
10.1. Integración electrónica final	11
11.Circuitos y conexiones	14
11.1. ATmega328P Maestro	14
11.2. Slave 1 – 0x11	14
11.3. Slave 2 – 0x12	15
11.4. Conexión del ESP32	15
12.Diagrama de flujo y pseudocódigo	15
13.Implementación del software	16
13.1. Programa Maestro	16
13.2. Slave 1	16
13.3. Slave 2	17
13.4. Protocolo de aplicación	17
13.5. Integración del ESP32	17
14.ESP32 y Adafruit IO	17
14.1. Enlace UART entre el Maestro y el ESP32	18
14.2. Formato de las tramas	18
14.3. Feeds implementados	19
14.4. Publicación de telemetría	19
14.5. Control manual desde Adafruit IO	19
15.Pruebas y resultados	20
16.Análisis de resultados	20

17.Problemas encontrados y soluciones	21
17.1. Verificación de la comunicación I2C	21
17.2. Reinicio del Maestro durante el accionamiento del motor DC	22
18.Conclusiones	22
19.Bibliografía	23
20.Anexos	24
20.1. Organización del código fuente	24
20.2. Video de demostración	24

1. Introducción

Los sistemas embebidos permiten integrar sensores, actuadores, interfaces de usuario y protocolos de comunicación para automatizar procesos físicos. En el Proyecto 1 de Electrónica Digital 2 se requiere diseñar e implementar una red de sensores formada por cuatro microcontroladores en total, utilizando tres ATmega328P y un ESP32. El sistema debe incorporar tres sensores distintos, de los cuales al menos uno debe utilizar comunicación I2C, además de una pantalla LCD, tres tipos de motores —DC, Stepper y Servo— y comunicación con la plataforma Adafruit IO mediante WiFi.

Para cumplir estos requerimientos se propone un *Car Wash Inteligente*, representado mediante una maqueta que automatiza el recorrido de un automóvil de juguete a través de diferentes etapas de un lavado. El proceso comienza con la detección del vehículo en la entrada y continúa con su desplazamiento por las estaciones del sistema. Durante el recorrido se utilizan sensores para detectar la presencia del automóvil, monitorear el nivel de agua y determinar la llegada del vehículo al final del trayecto. De forma complementaria, distintos actuadores permiten controlar la barrera de entrada, el movimiento del automóvil y el accionamiento de los cepillos.

La arquitectura utiliza un ATmega328P como microcontrolador Maestro y dos ATmega328P como Slaves conectados mediante un bus I2C compartido. El Maestro también controla una pantalla LCD de 16x2 y se comunica con un sensor de distancia VL53L0X mediante I2C. Las mediciones y estados relevantes son transferidos al ESP32 mediante UART. El ESP32 implementa la conexión WiFi y el enlace con Adafruit IO, permitiendo publicar telemetría y, en modo manual, enviar órdenes de control hacia el Maestro.

El presente informe describe progresivamente el diseño, construcción e implementación del sistema. Las secciones de pruebas, resultados y análisis se completarán a partir de los datos obtenidos durante la implementación real, evitando incorporar resultados o mediciones no verificadas.

2. Objetivos

2.1. Objetivo general

Diseñar e implementar una maqueta de un Car Wash Inteligente basada en una red de sensores y actuadores controlada por tres microcontroladores ATmega328P y un ESP32, capaz de automatizar el recorrido de un automóvil de juguete, monitorear variables del proceso, visualizar información localmente y transmitir las mediciones hacia Adafruit IO mediante WiFi.

2.2. Objetivos específicos

- Implementar una red I2C entre un ATmega328P Maestro, dos ATmega328P Slaves y al menos un sensor con interfaz I2C.
- Integrar tres sensores de distinto tipo con funciones específicas dentro del proceso: detección de presencia, monitoreo del nivel de agua y detección de la posición final del automóvil.
- Controlar tres tipos de motores requeridos por el proyecto: un motor DC, un motor paso a paso y servomotores empleados en mecanismos del proceso.
- Mostrar desde el microcontrolador Maestro información relevante del sistema en una pantalla LCD de 16x2.
- Coordinar el funcionamiento simultáneo de sensores, actuadores y comunicaciones para ejecutar automáticamente la secuencia del lavado.
- Transferir las variables de medición al ESP32 mediante el protocolo de comunicación definido en la implementación final y enviarlas en tiempo real a Adafruit IO mediante WiFi.
- Diseñar y construir una maqueta en MDF que permita montar los sensores y actuadores y guiar el desplazamiento del automóvil durante el proceso.

3. Descripción general del proyecto

El proyecto consiste en una maqueta funcional de un lavado de autos automatizado. Un automóvil de juguete recorre longitudinalmente una estructura diseñada para representar las etapas principales del proceso. El movimiento del vehículo se realiza mediante un motor paso a paso 28BYJ-48 ubicado debajo de la base, el cual enrolla un hilo conectado al automóvil para desplazarlo a través de la maqueta.

En la entrada, un sensor infrarrojo detecta la presencia del vehículo. Esta detección permite iniciar la secuencia y controlar la apertura de una talanquera mediante un servomotor. Durante el recorrido, un sensor ultrasónico HC-SR04 mide la distancia entre su posición y la superficie del agua de un depósito, proporcionando una variable asociada al nivel disponible. En otra estación, un motor DC acciona el mecanismo de cepillado. Finalmente, un sensor VL53L0X con comunicación I2C detecta la llegada del automóvil a la zona final del recorrido.

El control se distribuye entre tres ATmega328P. El Maestro concentra la interfaz LCD y el bus I2C, mientras que los dos Slaves controlan sensores y actuadores ubicados en distintas zonas del proceso. El sistema incorpora además un ESP32 para la etapa de conectividad con Adafruit IO. La distribución detallada de entradas y salidas se presenta posteriormente en la sección de arquitectura y conexiones.

La maqueta fue diseñada en Autodesk Inventor y está planteada para fabricarse principalmente en MDF mediante corte láser. Su estructura es abierta, con paredes laterales y soportes localizados para los componentes que requieren montaje sobre el recorrido. El diseño también contempla una caja inferior para alojar y proteger el mecanismo del motor paso a paso.

4. Funcionamiento del sistema

La secuencia de operación prevista para el Car Wash Inteligente es la siguiente:

1. El sistema permanece en estado de espera hasta detectar un automóvil en la entrada.
2. El sensor infrarrojo informa la presencia del vehículo.
3. El servomotor de la puerta abre la talanquera de entrada.
4. El motor Stepper comienza a enrollar el hilo conectado al automóvil y lo desplaza a lo largo de la maqueta.
5. En la zona de agua, el HC-SR04 mide la distancia hasta la superficie del líquido para monitorear el nivel del depósito.
6. El automóvil continúa hacia la estación de cepillado, donde el motor DC acciona los cepillos durante la etapa correspondiente del proceso.
7. El vehículo continúa avanzando hacia la salida.
8. El sensor VL53L0X detecta la llegada del automóvil a la posición final.
9. El sistema detiene el desplazamiento y considera finalizado el ciclo de lavado.
10. Las variables y estados relevantes se muestran en la LCD y pueden enviarse al ESP32 mediante UART para su publicación en Adafruit IO. El sistema incorpora además un modo manual en el que Adafruit IO puede enviar órdenes de control hacia los actuadores a través del Maestro.

5. Arquitectura del sistema y distribución de micro-controladores

La distribución actual del sistema utiliza tres ATmega328P conectados mediante un bus I2C compartido. El Maestro coordina la red, controla la pantalla LCD y se comunica directamente con el VL53L0X. Los dos Slaves tienen direcciones I2C distintas y concentran sensores y actuadores de diferentes zonas del proceso.

5.1. ATmega328P Maestro

- UART por PD0/RX y PD1/TX, configurada actualmente a 9600 baudios, 8N1.
- LCD de 16x2 en modo de 8 bits: D0–D7 conectados desde PD2 hasta PB1; E en PC0 y RS en PC1.
- Bus I2C: SDA en PC4/A4 y SCL en PC5/A5.
- VL53L0X (Slave 3) en el bus I2C, con dirección 0x29.
- XSHUT del VL53L0X conectado a PB2/D10.

5.2. ATmega328P Slave 1 – dirección I2C 0x11

- HC-SR04: ECHO en PC2/A2 y TRIG en PC3/A3.
- Servo de agua en PB3/D11, con posiciones actuales de 0 grados para cerrado y 90 grados para abierto.
- L298N para el motor DC: ENA en PB1/D9, IN1 en PD7/D7 e IN2 en PB0/D8.
- PWM de lavado indicado actualmente: 128.

5.3. ATmega328P Slave 2 – dirección I2C 0x12

- Sensor infrarrojo en PC0/A0, activo en nivel bajo.
- ULN2003 del Stepper 28BYJ-48: IN1–IN4 conectados a PD2/D2, PD3/D3, PD4/D4 y PD5/D5.
- Servo de puerta en PD6/D6, con posiciones actuales de 145 grados para cerrado y 50 grados para abierto.

- Movimiento del Stepper mediante medio paso, con velocidad indicada actualmente de 800 pasos por segundo.

5.4. Bus I2C compartido

SDA conecta A4 del Maestro, A4 del Slave 1, A4 del Slave 2 y SDA del VL53L0X. SCL conecta de forma equivalente los pines A5 y SCL de los dispositivos. Se conectan resistencias de $470\ \Omega$ en configuración *pull-up* en las líneas SDA y SCL.

5.5. Alimentación

Los microcontroladores, sensores y señales de control comparten una referencia común de tierra. El motor DC utiliza una fuente independiente conectada a VS del L298N; los servomotores deben alimentarse mediante una fuente externa regulada de 5 V con capacidad de corriente suficiente; y el Stepper debe alimentarse a través del ULN2003 con una fuente adecuada. Todas las tierras deben compartir referencia, procurando evitar que las corrientes de los motores circulen por el cableado del bus I2C.

6. Red de sensores

La red de sensado está distribuida entre los tres ATmega328P. El Maestro solicita las mediciones de los sensores conectados a los Slaves mediante I2C y, además, accede directamente al sensor ToF VL53L0X. Esta distribución permite que cada microcontrolador atienda localmente sus dispositivos y que el Maestro concentre la toma de decisiones del proceso.

6.1. Sensor infrarrojo de entrada

El sensor infrarrojo está conectado al Slave 2 en PC0/A0. Su salida es activa en bajo: un nivel lógico bajo representa detección del vehículo y un nivel alto representa espacio libre. El Slave 2 actualiza la lectura aproximadamente cada 10 ms y mantiene un contador de muestras para que el Maestro pueda distinguir datos nuevos.

En estado de reposo, el Maestro consulta el sensor cada 100 ms. Para reducir activaciones causadas por cambios aislados, la lógica automática requiere cinco detecciones consecutivas antes de iniciar el ingreso del vehículo. Una vez confirmada la presencia, el Maestro ordena abrir la puerta de entrada.

6.2. HC-SR04 y nivel del depósito

El HC-SR04 está conectado al Slave 1, con ECHO en PC2/A2 y TRIG en PC3/A3. El Slave 1 realiza la adquisición local y entrega al Maestro el ancho del pulso de eco. El Maestro convierte posteriormente dicho tiempo a distancia.

Durante la etapa asociada al suministro de líquido, la primera medición válida se almacena como distancia de referencia. Después se abre el servo de agua a 90 grados y se continúa monitoreando el depósito. Cuando la distancia medida aumenta 20 mm respecto de la referencia, el sistema cierra el servo y reanuda el desplazamiento del automóvil. Por tanto, la medición ultrasónica no se utiliza únicamente para visualización, sino también como condición de control del proceso.

6.3. VL53L0X

El VL53L0X, identificado en la arquitectura como Slave 3, está conectado directamente al bus I2C del Maestro y utiliza la dirección 0x29. La línea XSHUT está conectada a PB2/D10 y permite mantener el sensor apagado o reiniciarlo físicamente. En la implementación actual, el Maestro inicializa el sensor cuando comienza el transporte del vehículo y realiza mediciones de distancia durante distintas etapas.

El código utiliza el VL53L0X como referencia de posición a lo largo del recorrido. La zona de suministro se identifica entre 265 y 270 mm, la estación de cepillado entre 125 y 130 mm, y la condición de finalización se detecta cuando la distancia es menor o igual a 70 mm. Las mediciones consideradas físicamente válidas por la lógica se encuentran entre 20 y 2000 mm. Estos umbrales corresponden a la implementación actual y deberán contrastarse con las pruebas finales de la maqueta.

7. Actuadores

El proceso utiliza los tres tipos de motores requeridos: servomotores para mecanismos de apertura, un motor DC para el cepillado y un motor paso a paso para el transporte del vehículo.

7.1. Servomotores

Se emplean dos servomotores. El servo de puerta está conectado al Slave 2 en PD6/D6. La posición programada para puerta cerrada es 145 grados y para puerta abierta es 50 grados.

El segundo servo se encuentra en el Slave 1, conectado a PB3/D11, y controla el suministro de líquido; utiliza 0 grados como posición cerrada y 90 grados como posición abierta. Las posiciones son solicitadas por el Maestro mediante comandos enviados por I2C.

7.2. Motor DC y L298N

El motor DC destinado al cepillado es controlado por el Slave 1 mediante un puente H L298N. ENA está conectado a PB1/D9, IN1 a PD7/D7 e IN2 a PB0/D8. Durante la fase de lavado, el Maestro solicita dirección hacia adelante y un valor PWM de 128. La fase de cepillado programada dura cinco segundos, tras los cuales el motor se detiene y el vehículo continúa hacia la salida.

7.3. Motor paso a paso y ULN2003

El transporte del automóvil se realiza mediante un Stepper 28BYJ-48 controlado por el Slave 2 a través de un ULN2003. Las entradas IN1–IN4 se conectan a PD2, PD3, PD4 y PD5. La librería se inicializa en modo de medio paso y la velocidad utilizada por la secuencia automática es de 800 pasos por segundo.

El Maestro puede ordenar movimiento continuo hacia adelante o en reversa. Durante el proceso normal, el Stepper avanza hasta alcanzar las ventanas de distancia definidas por el VL53L0X, se detiene durante las estaciones de proceso y vuelve a arrancar posteriormente. Al finalizar el ciclo, el programa ejecuta un movimiento en reversa durante cinco segundos antes de esperar el retiro del vehículo.

8. Pantalla LCD

La interfaz local utiliza una pantalla LCD 16x2 controlada directamente por el ATmega328P Maestro mediante una librería propia en modo paralelo de 8 bits. Las líneas D0–D5 se conectan a PD2–PD7, D6 a PB0 y D7 a PB1. La señal Enable se conecta a PC0/A0 y Register Select a PC1/A1; R/W se mantiene conectado a tierra para trabajar únicamente en escritura.

La pantalla presenta mensajes asociados al estado del proceso. Entre los textos implementados se encuentran “Bienvenido al / CarWash” durante el reposo, “Ingresando / tu auto” durante la entrada, “Enjabonando / tu auto” durante la etapa de suministro, “Lavando / tu auto” durante el cepillado y “Gracias por / elegirnos” al completar el recorrido. De esta forma, la LCD funciona como interfaz de estado para el usuario además de cumplir con el

requisito de visualización desde el microcontrolador Maestro.

9. Protocolos de comunicación

9.1. I2C

La comunicación principal entre los ATmega328P utiliza I2C. El Maestro configura el bus a 100 kHz y los Slaves responden en las direcciones 0x11 y 0x12. El VL53L0X, denominado Slave 3, comparte las mismas líneas SDA y SCL con dirección 0x29. SDA corresponde a PC4/A4 y SCL a PC5/A5 en los tres Arduino Nano.

Sobre I2C se implementó un protocolo propio de solicitudes y respuestas. El Maestro puede verificar la disponibilidad de los nodos, solicitar sensores y enviar órdenes a los actuadores. Los periféricos decodifican las solicitudes fuera de la rutina de interrupción TWI y publican respuestas que incluyen estados como operación correcta, solicitud aceptada, dispositivo no soportado, dispositivo desconocido o ausencia de una muestra válida. Esta estructura permite separar el transporte I2C de las acciones específicas del Car Wash.

El Maestro también comprueba periódicamente la comunicación con ambos Slaves. Durante la operación normal realiza una verificación aproximadamente cada 500 ms y entra a un estado de error si se pierde uno de los enlaces.

9.2. UART

El Maestro configura el USART del ATmega328P a 9600 baudios con formato de 8 bits de datos, sin paridad y un bit de parada. PD0/RX y PD1/TX quedan destinados a esta interfaz. La transmisión utiliza un buffer y la interrupción de registro de datos vacío, evitando que la impresión de mensajes bloquee directamente la lógica principal.

UART cumple dos funciones. La primera es diagnóstico y supervisión: el Maestro informa transiciones de estado, distancias, errores y permite solicitar reportes o ejecutar un escaneo del bus I2C. La segunda es el enlace con el ESP32. El Maestro transmite por PD1/D1 tramas ASCII de telemetría a 9600 baudios, mientras que las órdenes provenientes del ESP32 ingresan por PB4/D12 mediante una UART por software. En el ESP32 se utiliza UART2, con GPIO16 como RX y GPIO17 como TX.

Las tramas de telemetría siguen el formato @CW,T,CLAVE,VALOR, mientras que las órdenes remotas utilizan @CW,C,DISPOSITIVO,VALOR. Esta separación permite integrar la supervisión de Adafruit IO sin reemplazar el protocolo I2C utilizado entre los ATmega328P.

10. Diseño y construcción de la maqueta

La estructura física fue diseñada para corte láser y posteriormente ensamblada en MDF mediante uniones de pestañas y ranuras. El diseño distribuye el recorrido del automóvil en una base alargada con paredes laterales, zonas superiores para sostener los elementos de proceso y una caja inferior destinada al mecanismo de transporte.

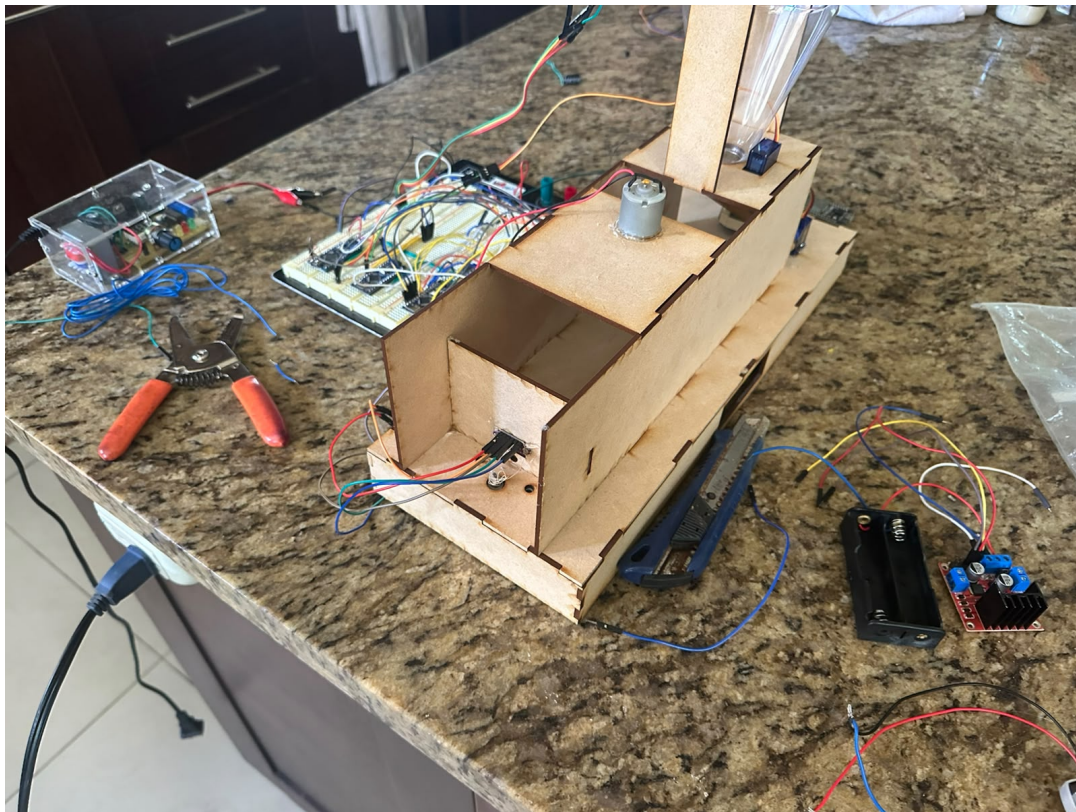


Figura 1: Vista general del prototipo físico del Car Wash ensamblado en MDF.



Figura 2: Entrada del Car Wash con vehículo, servomotor de puerta y sensor infrarrojo.

La entrada incorpora el vehículo de prueba, el servomotor que acciona la barrera y el módulo infrarrojo utilizado para detectar la presencia inicial. En la parte interior puede observarse el canal de recorrido y la abertura longitudinal asociada al mecanismo que desplaza el automóvil.

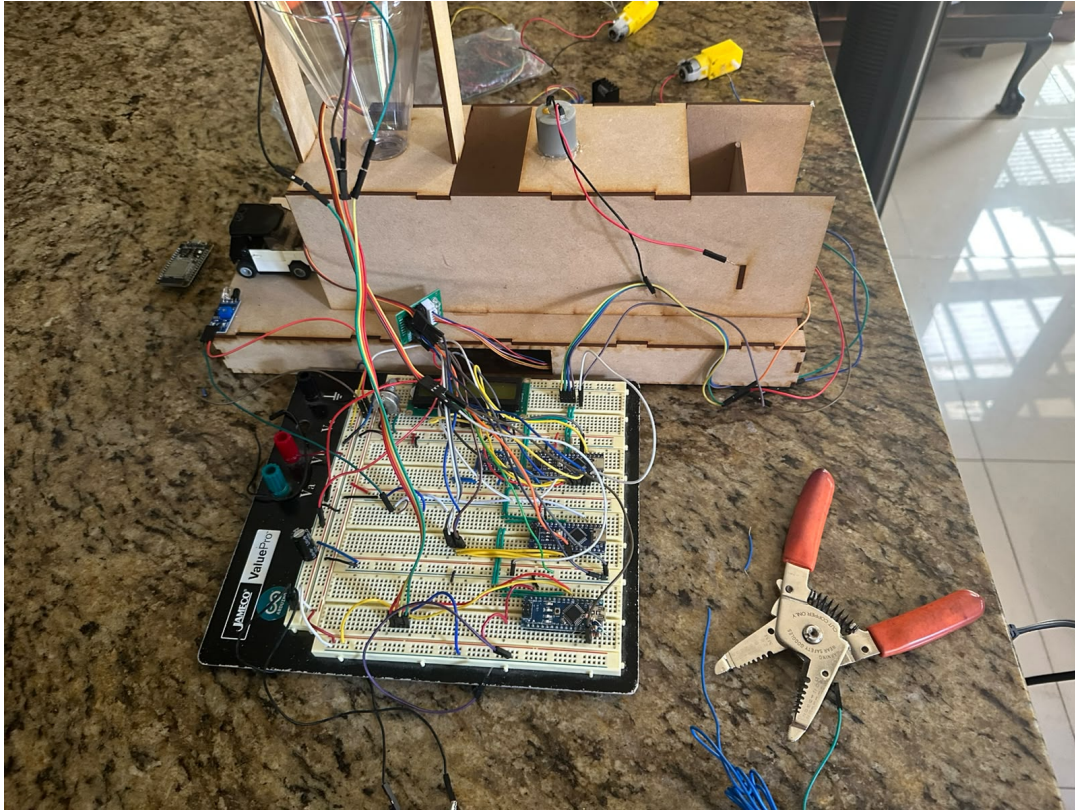


Figura 3: Prototipo conectado a la electrónica de control durante la etapa de integración.

10.1. Integración electrónica final

Durante la etapa final de integración, los tres Arduino Nano y el ESP32 fueron montados sobre la misma plataforma de prototipado junto con la pantalla LCD y el cableado de comunicación y alimentación. Esta disposición permitió verificar el sistema completo con los controladores conectados simultáneamente a la maqueta.

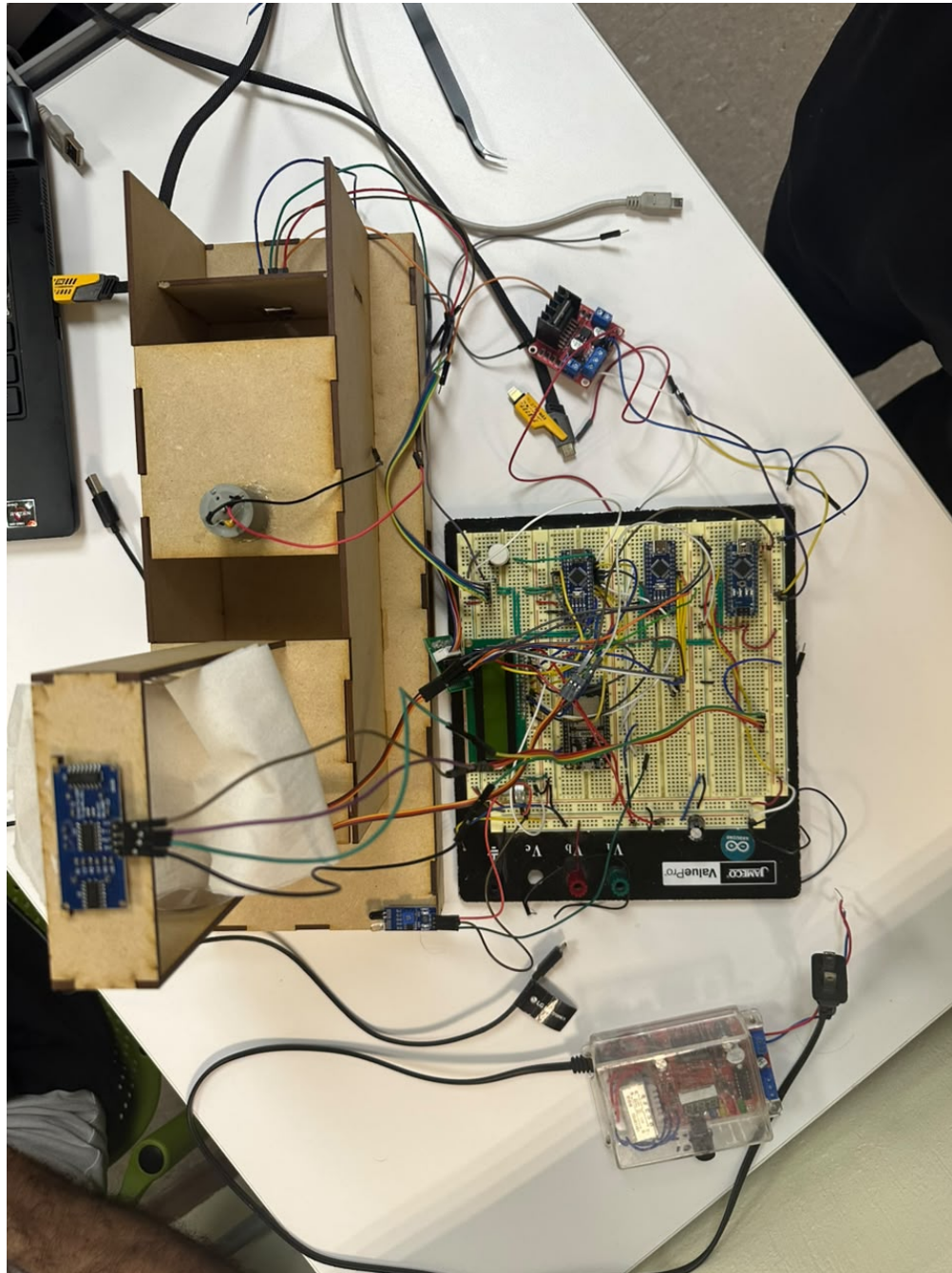


Figura 4: Vista superior de la maqueta y de la plataforma electrónica durante la integración final.

La siguiente fotografía permite observar con mayor detalle la distribución de los microcontroladores, el ESP32 y la pantalla LCD. El montaje se mantuvo en protoboard debido a que el proyecto se encontraba en etapa de pruebas e integración, facilitando modificaciones en las conexiones sin alterar la estructura mecánica.

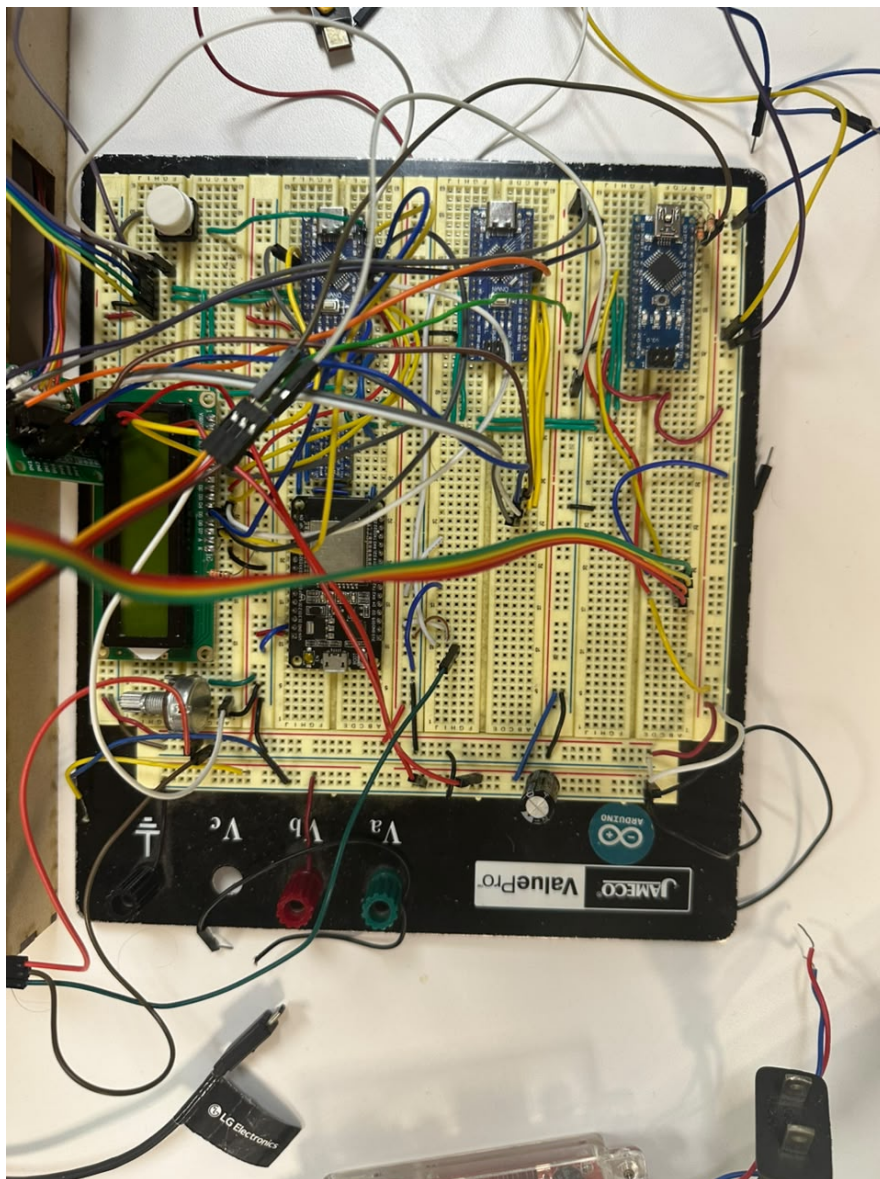


Figura 5: Detalle de la plataforma de control con los ATmega328P, ESP32 y LCD utilizados en el prototipo.

11. Circuitos y conexiones

11.1. ATmega328P Maestro

Pin Nano	Pin ATmega328P	Función
D0	PD0	RX UART
D1	PD1	TX UART
D2–D7	PD2–PD7	LCD D0–D5
D8	PB0	LCD D6
D9	PB1	LCD D7
D10	PB2	XSHUT VL53L0X
A0	PC0	Enable LCD
A1	PC1	RS LCD
A4	PC4	SDA I2C
A5	PC5	SCL I2C

Cuadro 1: Asignación principal de pines del Maestro.

11.2. Slave 1 – 0x11

Pin Nano	Pin ATmega328P	Función
A2	PC2	ECHO HC-SR04
A3	PC3	TRIG HC-SR04
A4	PC4	SDA I2C
A5	PC5	SCL I2C
D11	PB3	Servo de agua
D9	PB1	ENA L298N / PWM
D7	PD7	IN1 L298N
D8	PB0	IN2 L298N

Cuadro 2: Asignación principal de pines del Slave 1.

11.3. Slave 2 – 0x12

Pin Nano	Pin ATmega328P	Función
A0	PC0	Sensor infrarrojo
A4	PC4	SDA I2C
A5	PC5	SCL I2C
D2	PD2	ULN2003 IN1
D3	PD3	ULN2003 IN2
D4	PD4	ULN2003 IN3
D5	PD5	ULN2003 IN4
D6	PD6	Servo de puerta

Cuadro 3: Asignación principal de pines del Slave 2.

11.4. Conexión del ESP32

Cuadro 4: Señales utilizadas entre el Maestro y el ESP32.

Origen	Destino	Función
Master D1 / PD1 / TX	ESP32 GPIO16 / RX2	Telemetría hacia Adafruit IO
ESP32 GPIO17 / TX2	Master D12 / PB4	Órdenes desde Adafruit IO
GND Master	GND ESP32	Referencia común

12. Diagrama de flujo y pseudocódigo

1. Inicializar UART, LCD, I2C, VL53L0X y base de tiempo.
2. Comprobar comunicación con los dos Slaves I2C.
3. Colocar actuadores en estado seguro y mostrar el mensaje de bienvenida.
4. Consultar periódicamente el sensor IR hasta obtener cinco detecciones consecutivas.
5. Abrir la puerta de entrada y esperar 3 s.
6. Inicializar el VL53L0X y arrancar el Stepper hacia adelante a 800 pasos/s.
7. Cuando el VL53L0X se encuentre entre 265 y 270 mm, detener el Stepper y cerrar la puerta.
8. Tomar una referencia con el HC-SR04, abrir el servo de agua y monitorear la distancia.

9. Cuando la distancia ultrasónica aumente 20 mm, cerrar el agua y reanudar el Stepper.
10. Cuando el VL53L0X se encuentre entre 125 y 130 mm, detener el Stepper y encender el motor DC con PWM 128.
11. Mantener el cepillado durante 5 s, detener el motor DC y reanudar el Stepper.
12. Cuando el VL53L0X mida 70 mm o menos, detener el proceso, asegurar actuadores y apagar el VL53L0X.
13. Mover el Stepper en reversa durante 5 s y después detenerlo.
14. Esperar a que el sensor IR indique que el vehículo fue retirado y regresar al estado de reposo.
15. Durante los estados normales, comprobar periódicamente los enlaces I2C; ante un fallo, pasar al manejo de error y recuperación.

13. Implementación del software

El software está dividido en tres proyectos principales para ATmega328P: Maestro, Slave 1 y Slave 2. Cada proyecto utiliza librerías propias para separar el manejo de hardware de la lógica de aplicación.

13.1. Programa Maestro

El Maestro concentra la máquina de estados del Car Wash. Inicializa la LCD, UART, I2C, la base de tiempo y el VL53L0X. La aplicación evita estructurar el proceso completo mediante retardos largos; en su lugar utiliza marcas de tiempo y estados para continuar atendiendo comunicaciones y verificaciones de enlace mientras avanza la secuencia.

Los estados implementados cubren comprobación inicial, reposo, entrada, búsqueda de la estación de suministro, referencia y control del líquido, búsqueda y ejecución del lavado, búsqueda de salida, retorno final y estados de error/recuperación. El Maestro también mantiene una comprobación periódica de los Slaves y aplica acciones seguras si detecta una pérdida de comunicación.

13.2. Slave 1

El Slave 1 inicializa el HC-SR04, el servo de agua, el L298N y el módulo I2C en dirección 0x11. Su ciclo principal alterna la atención de solicitudes I2C con la actualización de la

medición ultrasónica. El nodo no decide por sí mismo la secuencia global: recibe del Maestro las órdenes de posición del servo y control del motor DC.

13.3. Slave 2

El Slave 2 inicializa el sensor infrarrojo, el Stepper en modo de medio paso, el servo de puerta y el módulo I2C en dirección 0x12. La lectura infrarroja se actualiza localmente y el movimiento del Stepper es administrado mediante su librería, permitiendo que el nodo continúe atendiendo solicitudes I2C mientras el motor se encuentra activo.

13.4. Protocolo de aplicación

Además de la librería de bajo nivel para I2C, los tres proyectos comparten una librería de protocolo. Esta define comandos para lectura de sensores, posicionamiento de servos, control del motor DC, movimiento del Stepper, parada y verificación de dispositivos. Las respuestas incluyen códigos de estado que permiten al Maestro detectar solicitudes inválidas, Slaves no disponibles y condiciones de comunicación anormales.

13.5. Integración del ESP32

La versión final del proyecto incluye el programa del ESP32 basado en Arduino y la librería `AdafruitIO_WiFi`. El programa mantiene simultáneamente la conexión con Adafruit IO y la recepción UART desde el Maestro. Las tramas recibidas se interpretan por clave, se almacenan en una estructura local y se publican de manera escalonada.

En el Maestro se añadió además la librería `ESPUART`, que implementa una recepción UART por software en PB4/D12 mediante interrupción por cambio de pin y Timer1. El botón conectado a PB3/D11 permite alternar entre la máquina de estados automática y un modo manual destinado al control desde Adafruit IO. En modo manual el Maestro valida rangos, envía las órdenes a Slave 1 o Slave 2 mediante el protocolo I2C y solamente actualiza la telemetría del actuador cuando la orden ha sido aceptada.

14. ESP32 y Adafruit IO

La versión final del código incorpora un ESP32 como puente entre el microcontrolador Maestro y la plataforma Adafruit IO. El ESP32 utiliza la librería `AdafruitIO_WiFi` para

establecer la conexión WiFi y administrar los feeds. Las credenciales de red y la clave privada de Adafruit IO se mantienen fuera de la documentación por razones de seguridad.

14.1. Enlace UART entre el Maestro y el ESP32

El enlace con el ESP32 trabaja a 9600 baudios y formato 8N1. En el ESP32 se utiliza `HardwareSerial(2)`, con GPIO16 como RX y GPIO17 como TX. La información enviada por el Maestro sale mediante su USART de hardware por PD1/D1 y es recibida por GPIO16 del ESP32. Para las órdenes en sentido contrario, GPIO17 del ESP32 se conecta al receptor implementado por software en PB4/D12 del Maestro.

Esta separación permite conservar el USART principal del ATmega328P para la transmisión de telemetría y mensajes, mientras que las órdenes provenientes del ESP32 se reciben mediante una rutina UART por software basada en interrupción por cambio de pin y Timer1. El receptor utiliza un buffer circular de 64 bytes y valida los ocho bits de datos y el bit de parada.

Debido a que el ATmega328P trabaja con lógica de 5 V y el ESP32 con lógica de 3.3 V, la conexión debe incorporar la adaptación de nivel correspondiente, particularmente en la señal transmitida desde el Maestro hacia el ESP32.

14.2. Formato de las tramas

La comunicación utiliza líneas ASCII terminadas en salto de línea. Para telemetría, el Maestro genera tramas con la estructura:

`@CW,T,CLAVE,VALOR`

El ESP32 identifica la clave, conserva el valor más reciente y marca el feed correspondiente como pendiente de publicación. Para órdenes desde Adafruit IO hacia el Maestro se utiliza:

`@CW,C,DISPOSITIVO,VALOR`

El Maestro solamente acepta estas órdenes cuando se encuentra en modo manual. De esta manera se evita que un cambio remoto interfiera directamente con la máquina de estados automática.

14.3. Feeds implementados

El programa del ESP32 define ocho feeds asociados a variables del Car Wash. La Tabla 5 resume su correspondencia.

Cuadro 5: Feeds implementados para Adafruit IO.

Clave UART	Feed	Información asociada
DC	<code>digital-2.dc</code>	Motor DC / valor firmado de control
HCR	<code>digital-2.hcr</code>	Distancia del HC-SR04
IR	<code>digital-2.ir</code>	Estado del sensor infrarrojo
LCD	<code>digital-2.lcd</code>	Estado o mensaje del sistema
WA	<code>digital-2.servo-agua</code>	Ángulo del servo de agua
DO	<code>digital-2.servo-puerta</code>	Ángulo del servo de puerta
ST	<code>digital-2.stepper</code>	Velocidad y dirección del Stepper
VL	<code>digital-2.vl53l0x</code>	Distancia medida por el VL53L0X

Además de estas claves, el Maestro transmite **MODE** para indicar si se encuentra en modo automático o manual. Esta variable es utilizada internamente por el ESP32 para decidir si debe aceptar y reenviar órdenes de control.

14.4. Publicación de telemetría

El ESP32 recibe continuamente las líneas del Maestro sin detener la ejecución de `io.run()`. Cuando un valor cambia, se almacena localmente y se marca como pendiente. La publicación se realiza de manera rotativa, enviando como máximo un feed pendiente aproximadamente cada 2200 ms. Esta estrategia evita concentrar múltiples publicaciones consecutivas y reduce la frecuencia de actualización hacia el servicio.

El código también contempla la pérdida y recuperación de la conexión con Adafruit IO. Cuando se detecta una nueva conexión, el ESP32 solicita los valores actuales de los feeds que permiten control para sincronizar el estado de la interfaz.

14.5. Control manual desde Adafruit IO

El Maestro incorpora un botón en PB3/D11 que alterna entre modo automático y modo manual con un antirrebote de 40 ms. Al entrar al modo manual se detienen los actuadores, se apaga temporalmente el VL53L0X y la LCD muestra *Modo manual / Adafruit IO*. Posteriormente el sistema permite recibir órdenes remotas para:

- Motor DC: valores entre -255 y 255. El signo determina la dirección y la magnitud determina el PWM.
- Servo de agua: ángulos entre 0 y 180 grados.
- Servo de puerta: ángulos entre 0 y 180 grados.
- Stepper: valores entre -1000 y 1000 pasos por segundo; el signo determina la dirección.
- Parada general: al escribir **STOP** en el feed asociado a la LCD se solicita la detención segura de los actuadores.

Mientras el sistema permanece en modo manual, el Maestro actualizaría también los sensores para visualizarlos en Adafruit IO. El HC-SR04 y el sensor infrarrojo se consultan alternadamente cada 250 ms, mientras que el VL53L0X se mantiene mediante su rutina no bloqueante. Un valor de -1 se utiliza cuando no existe una muestra válida o cuando no se logra obtener temporalmente la medición correspondiente.

15. Pruebas y resultados

Primero se verificaron individualmente los sensores y actuadores desde el main de cada Slave. Después se integró la comunicación I2C, logrando que el Maestro ejecutara correctamente las acciones mediante ambos Slaves.

La secuencia completa funcionó, excepto al activar el motor DC, cuyo ruido eléctrico provocó inestabilidad y reinicios. Se probaron distintos puentes H y las recomendaciones del profesor, pero no se logró eliminar completamente el problema. Sin el motor conectado, el sistema finaliza correctamente.

La conexión con Adafruit IO no pudo establecerse durante las pruebas. Se consideró como posible causa que el punto de acceso del teléfono operara en la banda de 5 GHz. Sin embargo, esta hipótesis no fue comprobada experimentalmente.

16. Análisis de resultados

Como se puede observar en el video de demostración, el sensor VL53L0X envía correctamente las lecturas de distancia al Maestro. Estas mediciones son interpretadas para identificar los rangos establecidos y ejecutar las secuencias programadas en cada estación del Car Wash.

La implementación de mensajes de estado y detección de errores en la comunicación I2C resultó útil durante el desarrollo. El Maestro permitió identificar qué Slave había perdido la conexión, detener el proceso de manera segura y evitar que la secuencia continuara con información inválida. Esto facilitó la depuración de errores de programación, cableado y comunicación.

En conjunto, las pruebas demostraron que la arquitectura distribuida permitió separar correctamente las funciones del sistema. Los Slaves obtienen datos y ejecutan instrucciones, mientras que el Maestro interpreta las mediciones, detecta fallas y controla la secuencia general. Sin el motor DC conectado, el sistema pudo completar correctamente el recorrido programado.

17. Problemas encontrados y soluciones

17.1. Verificación de la comunicación I2C

Durante la integración de la red I2C se presentó dificultad para determinar con certeza si los Slaves se encontraban operando correctamente y si las solicitudes enviadas por el Maestro estaban siendo recibidas y aceptadas. Esta condición complicaba el diagnóstico, ya que una ausencia de respuesta podía deberse tanto a un problema de conexión como a una solicitud no reconocida o a la falta de una muestra válida.

Como medida de diagnóstico y robustez se incorporaron estados de respuesta dentro del protocolo de comunicación. Estos permiten distinguir entre una operación correcta, una solicitud aceptada, un dispositivo no soportado, un dispositivo desconocido y la ausencia de una muestra válida. De esta manera, el Maestro dispone de información adicional para identificar el estado de las transacciones y diferenciar distintos tipos de falla durante la ejecución.

Adicionalmente, se implementó una verificación inicial de la comunicación I2C con los Slaves. Durante esta etapa se comprueba la presencia de cada dispositivo utilizando su dirección correspondiente antes de comenzar la secuencia principal del Car Wash. Esta verificación facilitó la detección de problemas de conexión y permitió confirmar que los nodos esperados se encontraban disponibles en el bus antes de iniciar el proceso.

17.2. Reinicio del Maestro durante el accionamiento del motor DC

Durante las pruebas del sistema se observó un problema asociado al accionamiento del motor DC de la estación de cepillado. Con el motor conectado, el microcontrolador Maestro llega a reiniciarse durante el proceso y, como consecuencia, la secuencia no alcanza la lectura final necesaria para declarar terminado el ciclo de lavado. Al desconectar el motor DC, el sistema sí logra continuar la secuencia y finalizar el proceso.

A partir de este comportamiento se plantea como hipótesis que el accionamiento del motor introduce perturbaciones eléctricas sobre la alimentación o la referencia común del sistema. Estas perturbaciones pueden estar relacionadas con transitorios, ruido eléctrico o efectos parasitarios producidos por la naturaleza inductiva del motor y por los cambios de corriente durante su conmutación. Una caída momentánea de la tensión de alimentación o una perturbación suficientemente grande podría provocar el reinicio del microcontrolador como mecanismo de protección o por activación de las condiciones de reset del sistema.

18. Conclusiones

- Se logró implementar una maqueta de Car Wash basada en una arquitectura distribuida con tres ATmega328P y un ESP32. El sistema integró sensores, actuadores, una pantalla LCD y diferentes protocolos de comunicación para ejecutar la mayor parte del proceso automático.
- Se implementó exitosamente la red I2C entre el Maestro, los dos Slaves y el VL53L0X. El Maestro pudo solicitar mediciones, enviar instrucciones y detectar errores o desconexiones en el bus.
- Se integraron tres sensores con funciones específicas: el sensor IR detectó la presencia del vehículo, el HC-SR04 permitió controlar la etapa de suministro de agua y el VL53L0X determinó la posición del automóvil durante el recorrido.
- El VL53L0X transmitió correctamente sus mediciones al Maestro. Como se observa en el video, estas lecturas fueron interpretadas para reconocer los rangos establecidos y ejecutar las diferentes etapas del Car Wash.
- Se controlaron los tres tipos de motores solicitados: servomotores para la puerta y el suministro de agua, un motor paso a paso para desplazar el vehículo y un motor DC para el cepillado. Sin embargo, el motor DC presentó problemas de ruido eléctrico que impidieron validar de forma estable su funcionamiento dentro de la secuencia completa.

- La pantalla LCD permitió mostrar el estado actual del proceso y mensajes de diagnóstico, proporcionando al usuario información local sobre el funcionamiento del sistema.
- La máquina de estados del Maestro permitió coordinar los sensores, actuadores y tiempos del proceso. Sin el motor DC conectado, el sistema logró completar la secuencia automática y alcanzar correctamente el estado de finalización.
- La identificación de desconexiones y fallas I2C facilitó la depuración del sistema, ya que permitió determinar qué dispositivo había perdido comunicación y corregir problemas de programación o cableado.
- Se implementó el enlace UART entre el Maestro y el ESP32, junto con el código necesario para enviar telemetría y recibir instrucciones desde Adafruit IO. No obstante, la conexión con la plataforma no pudo validarse experimentalmente, por lo que este objetivo se alcanzó únicamente a nivel de implementación de software.

19. Bibliografía

Referencias

- [1] STMicroelectronics, *VL53L0X: Time-of-Flight Ranging Sensor, Datasheet DS11555, Rev. 6*. Disponible en: <https://www.st.com/resource/en/datasheet/vl53l0x.pdf>
- [2] SparkFun Electronics, *HC-SR04 Ultrasonic Distance Sensor Datasheet*. Disponible en: <https://www.digikey.com/en/htmldatasheets/production/3822706/0/0/1/hc-sr04.html>
- [3] Microchip Technology, *ATmega328P: 8-bit AVR Microcontroller Datasheet*. Disponible en: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
- [4] NXP Semiconductors, *UM10204: I2C-bus Specification and User Manual, Rev. 7*. Disponible en: <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>
- [5] STMicroelectronics, *L298: Dual Full-Bridge Driver Datasheet*. Disponible en: <https://www.st.com/resource/en/datasheet/l298.pdf>
- [6] Texas Instruments, *ULN2002A, ULN2003A and ULN2003AI Darlington Transistor Arrays Datasheet*. Disponible en: <https://www.ti.com/lit/ds/symlink/uln2003a.pdf>

- [7] Espressif Systems, *ESP32 Series Datasheet*. Disponible en: https://documentation.espressif.com/esp32_datasheet_en.pdf
- [8] Espressif Systems, *Arduino ESP32: Serial UART Documentation*. Disponible en: <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/serial.html>
- [9] Adafruit Industries, *Adafruit IO Arduino Client Library Documentation*. Disponible en: https://adafruit.github.io/Adafruit_IO_Arduino/html/index.html
- [10] Adafruit Industries, *Adafruit IO Basics: Feeds*. Disponible en: <https://learn.adafruit.com/adafruit-io-basics-feeds/overview>
- [11] AVR Libc Project, *AVR Libc Reference Manual*. Disponible en: <https://avrdudes.github.io/avr-libc/avr-libc-user-manual/>

20. Anexos

Enlace a GitHub: Repositorio del proyecto Car Wash

20.1. Organización del código fuente

El proyecto completo se encuentra separado en cuatro aplicaciones: Maestro ATmega328P, Slave 1, Slave 2 y ESP32. Los proyectos de los ATmega328P incluyen librerías propias para I2C, protocolo, sensores y actuadores. El Maestro incorpora además LCD, VL53L0X, base de tiempo, UART y la recepción UART por software desde el ESP32.

El código del ESP32 se encuentra en el archivo `implementacionadafruit.ino` y contiene la integración con `AdafruitIO_WiFi`, `UART2` y los ocho feeds descritos en este informe.

Los archivos completos de código se entregan en el repositorio del proyecto; en el cuerpo del informe se presentan únicamente las estructuras y fragmentos necesarios para explicar la implementación.

20.2. Video de demostración

Como evidencia adicional del funcionamiento e integración del prototipo se registró un video de demostración y se publicó en YouTube como parte de los entregables del proyecto.

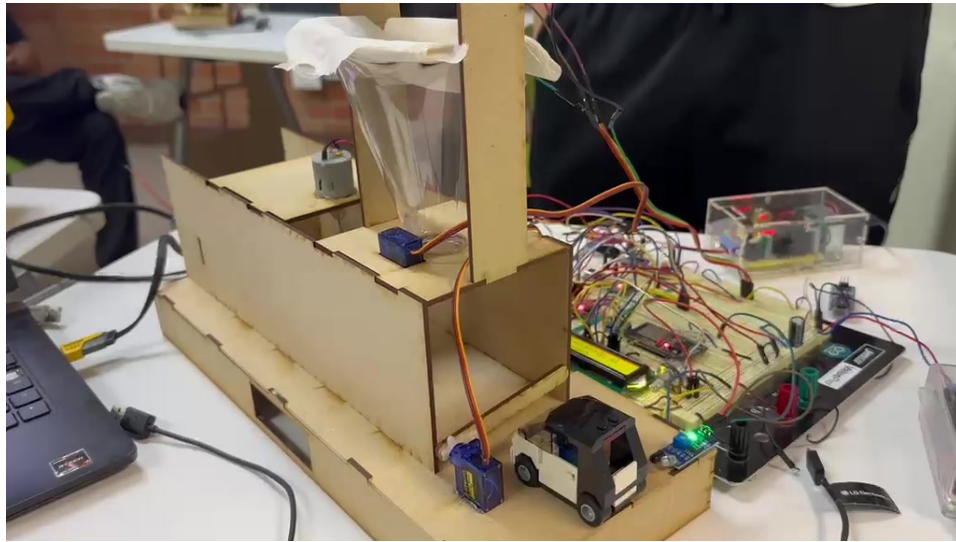


Figura 6: Fotograma del video de demostración del prototipo durante las pruebas de funcionamiento.

El video de demostración se encuentra disponible en YouTube en el siguiente enlace: <https://youtu.be/x-vYncTifNw>.

Enlace a github: <https://github.com/DonMiguelMatta/Digital-2/tree/main/ProyectoI2C>.